



پروژه درس میکروپروسسور

استاد: دکتر یثربی

برنامه نویسی AVR

هستی نواب دانشمند

40232152

مقدمه:

در سؤال اول، یک صفحه کلید ماتریسی 4×4 به پورت A میکروکنترلر ATmega32 متصل شده است. برنامه باید بدون استفاده از سخت افزار جانبی، ردیف ها و ستون های صفحه کلید را اسکن کرده و کلید فشرده شده را تشخیص دهد. مقدار خام هر کلید با استفاده از یک جدول ذخیره شده در EEPROM به مقدار واقعی آن تبدیل می شود و سپس کلیدهای وارد شده از آدرس 0x0400 به بعد در حافظه SRAM ذخیره می شوند.

در این برنامه برای جلوگیری از ثبت چندباره یک کلید، پس از تشخیص آن تا زمان رهاشدن کلید صبر می شود. به همین دلیل، نگه داشتن یک کلید فقط باعث ثبت یک مقدار در بافر خواهد شد، اما کاربر می تواند با فشردن و رهاکردن متوالی کلیدها، یک عدد چندرقمی وارد کند. همچنین تابع Readkey برای خواندن آخرین کلید ذخیره شده از بافر نوشته شده است و در صورت خالی بودن بافر، مقدار 0xFF را بازمی گرداند.

در سؤال دوم، یک سیستم ثبت و نمایش مقدار اشباع اکسیژن خون با استفاده از دو میکروکنترلر ATmega32 پیاده سازی شده است. میکروکنترلر اصلی، فرمان کاربر را از همان صفحه کلید ماتریسی دریافت کرده و با ارسال پالس به میکروکنترلر سنسور، مقادیر اکسی هموگلوبین و دئوکسی هموگلوبین را دریافت می کند. اطلاعات دریافتی با عدد 0x34 رمزگشایی شده و سپس مقدار SpO₂ محاسبه می شود.

با وارد کردن فرمان A می تواند یک مقدار جدید را محاسبه کرده و در یکی از هشت خانه مشخص EEPROM ذخیره کند. با وارد کردن فرمان B نیز مقدار ذخیره شده از خانه انتخابی بازیابی می شود. نتیجه محاسبه یا بازیابی شده در نهایت به دو رقم دهگان و یکان تفکیک شده و روی دو سون سگمنت کاتد مشترک نمایش داده می شود.

۱. تعریف میکروکنترلر و بردار شروع برنامه

در ابتدای برنامه، فایل `m32def.inc` فراخوانی شده است تا نام رجیسترها، پلایه‌ها و ثلثت‌های مربوط به میکروکنترلر `ATmega32` در دسترس اسمبلر قرار گیرد. دستور `CSEG` شروع بخش حافظه برنامه را مشخص می‌کند. با قرار گرفتن دستور پرش در آدرس صفر، پس از روشن شدن یا ریست میکروکنترلر، اجرای برنامه مستقیماً از برچسب `start` آغاز می‌شود.

```
.INCLUDE "m32def.inc"
```

```
.CSEG
```

```
.ORG 0x0000
```

```
; IVT
```

```
RJMP start
```

۲. مقداردهی اولیه `Stack Pointer`

در ابتدای بخش `start`، اشاره‌گر پشته مقداردهی می‌شود. آدرس آخر حافظه `SRAM` که با `RAMEND` مشخص شده است، در دو رجیستر `SPH` و `SPL` قرار می‌گیرد. این کار برای اجرای صحیح دستورهای فراخوانی تابع مانند `RCALL` و بازگشت از تابع با `RET` ضروری است، زیرا آدرس بازگشت توابع در استک ذخیره می‌شود.

`start`:

```
; stack init.
```

```
LDI R16, HIGH(RAMEND)
```

```
OUT SPH, R16
```

```
LDI R16, LOW(RAMEND)
```

```
OUT SPL, R16
```

۳. تنظیم پورت A برای صفحه کلید ماتریسی

چهار بیت بالایی پورت A، یعنی پایه‌های PA4 تا PA7، به عنوان خروجی و چهار بیت پایینی به عنوان ورودی تنظیم می‌شوند. پایه‌های خروجی برای فعال کردن ردیف‌های صفحه کلید و پایه‌های ورودی برای خواندن ستون‌ها استفاده می‌شوند. با قراردادن مقدار 0xFF در PORTA، مقاومت‌های Pull-up داخلی پایه‌های ورودی فعال می‌شوند.

; porta keypad

LDI R16, 0xF0

OUT DDRA, R16

LDI R16, 0xFF

OUT PORTA, R16

۴. تنظیم پورت B برای دریافت داده سنسور

تمام پایه‌های پورت B به عنوان ورودی تنظیم شده‌اند. میکروکنترلر سنسور، اطلاعات اکسی‌هموگلوبین و دئوکسی‌هموگلوبین را روی این پورت قرار می‌دهد و میکروکنترلر دیتالاگر آن‌ها را از رجیستر PINB می‌خواند. مقدار صفر در PORTB نیز باعث می‌شود مقاومت‌های Pull-up داخلی این پورت غیرفعال باقی بمانند.

; portb az sensor data migire

LDI R16, 0x00

OUT DDRB, R16

OUT PORTB, R16

۵. تنظیم پورت C برای یکان و تحریک INT0 سنسور

تمام پایه‌های پورت C به‌عنوان خروجی تنظیم شده‌اند. پایه‌های PC0 تا PC6 برای کنترل سون‌سگمنت رقم یکان استفاده می‌شوند. پایه PC7 نیز به ورودی INT0 میکروکنترلر سنسور متصل است و با ایجاد لبه بالارونده، سنسور را برای ارسال مقدار دئوکسی‌هموگلوبین فعال می‌کند. در شروع برنامه تمام پایه‌ها صفر می‌شوند.

; port c

```
LDI R16, 0xFF
```

```
OUT DDRC, R16
```

```
LDI R16, 0x00
```

```
OUT PORTC, R16
```

۶. تنظیم پورت D برای دهگان و تحریک INT1 سنسور

تمام پایه‌های پورت D نیز به‌عنوان خروجی تعریف شده‌اند. پایه‌های PD0 تا PD6 برای کنترل سون‌سگمنت رقم دهگان به کار می‌روند. پایه PD7 به ورودی INT1 میکروکنترلر سنسور متصل است و برای درخواست مقدار اکسی‌هموگلوبین استفاده می‌شود. در ابتدای برنامه این پورت نیز صفر می‌شود.

; port d

```
LDI R16, 0xFF
```

```
OUT DDRD, R16
```

```
LDI R16, 0x00
```

```
OUT PORTD, R16
```

۷. پاک کردن فلگ T

در این برنامه از بیت T موجود در رجیستر وضعیت برای تشخیص پایان فرمان استفاده شده است. هنگامی که کاربر کلید F را فشار می‌دهد، این بیت یک می‌شود. در ابتدای برنامه با دستور CLT این بیت پاک می‌شود تا میکروکنترلر در حالت دریافت فرمان جدید قرار گیرد.

; flag

CLT

۸. تعیین محل اولیه بافر در SRAM

رجیستر Z از ترکیب دو رجیستر ZH و ZL تشکیل شده است. در این قسمت مقدار 0x0400 در رجیستر Z قرار داده می‌شود تا به ابتدای بافر ذخیره کلیدها در SRAM اشاره کند. کلیدهای A یا B و شماره خانه، پیش از دریافت کلید F، در این ناحیه ذخیره می‌شوند.

; mahal avalie buffer

LDI ZH, HIGH(0x0400)

LDI ZL, LOW(0x0400)

۹. حلقه اصلی برنامه

در ابتدای حلقه اصلی، وضعیت بیت T بررسی می‌شود. اگر این بیت یک باشد، به این معنا است که کاربر کلید F را فشار داده و فرمان کامل شده است؛ بنابراین برنامه به بخش Ready می‌رود. در غیر این صورت تابع صفحه کلید فراخوانی می‌شود. اگر هیچ کلیدی فشرده نشده باشد، مقدار 0xFF بازگردانده شده و حلقه مجدداً اجرا می‌شود.

Main:

BRTS Ready

RCALL Keypad

CPI R16, 0xFF

BREQ Main

۱۰. تبدیل مقدار خام کلید با استفاده از EEPROM

مقداری که از صفحه کلید دریافت می‌شود، مستقیماً عدد یا حرف کلید نیست، بلکه ترکیبی از وضعیت ردیف و ستون است. این مقدار خام در رجیستر R16 قرار دارد. تابع E2prom مقدار خام را به عنوان آدرس EEPROM در نظر می‌گیرد و کد واقعی کلید را از آن آدرس می‌خواند.

RCALL E2prom

۱۱. تشخیص کلید F و ذخیره سایر کلیدها

پس از تبدیل مقدار خام، برنامه بررسی می‌کند که آیا کلید فشرده شده F است یا خیر. کد کلید F برابر 0x0F است. اگر کلید F نباشد، مقدار کلید با استفاده از رجیستر Z در بافر SRAM ذخیره شده و اشاره گر یک واحد افزایش پیدا می‌کند. سپس برنامه وارد بخش انتظار برای رهاسدن کلید می‌شود.

; akharin kilid F boode ya na?

CPI R16, 0x0F

BREQ KeyF

ST Z+, R16

RJMP Wait

۱۲. بخش تشخیص کلید F

هنگامی که کلید F تشخیص داده شود، برنامه به برچسب KeyF منتقل می‌شود. در این قسمت دستور SET بیت T رجیستر وضعیت را یک می‌کند. کلید F در بافر ذخیره نمی‌شود و فقط به عنوان علامت پایان فرمان عمل می‌کند. در اجرای بعدی حلقه اصلی، برنامه با مشاهده بیت T وارد بخش پردازش فرمان می‌شود.

KeyF:

SET

۱۳. انتظار برای رهاشدن کلید

پس از تشخیص هر کلید، برنامه باید صبر کند تا کاربر دست خود را از روی کلید بردارد. در غیر این صورت، یک کلید ممکن است چند بار پشت سر هم در بافر ذخیره شود. تابع Keypad تا زمانی فراخوانی می‌شود که مقدار 0xFF، به معنای عدم فشردن کلید، دریافت شود.

Wait:

RCALL Keypad

CPI R16, 0xFF

BRNE Wait

RJMP Main

۱۴. استخراج اطلاعات فرمان از بافر

بعد از فشردن کلید F، برنامه وارد بخش Ready می‌شود. تابع Readkey ابتدا آخرین مقدار ذخیره شده، یعنی شماره خانه، را از بافر می‌خواند و در R18 قرار می‌دهد. بار دوم، نوع فرمان یعنی A یا B خوانده شده و در R19 ذخیره می‌شود. سپس بیت T پاک می‌شود تا برنامه برای فرمان بعدی آماده باشد.

Ready:

RCALL Readkey

MOV R18, R16

RCALL Readkey

MOV R19, R16

CLT

۱۵. تشخیص فرمان A یا B

نوع فرمان ذخیره شده در R19 با کدهای A و B مقایسه می شود. کد کلید A برابر 0x0A و کد کلید B برابر 0x0B است. اگر فرمان A باشد، نمونه جدید از سنسور دریافت و ذخیره می شود. اگر فرمان B باشد، مقدار قبلی از EEPROM خوانده خواهد شد. سایر مقادیر نامعتبر در نظر گرفته شده و برنامه به حلقه اصلی برمی گردد.

; A

CPI R19, 0x0A

BREQ A

; B

CPI R19, 0x0B

BREQ B

; na A na B

RJMP Main

۱۶. ایجاد پالس برای دریافت دثوکسی هموگلوبین

در ابتدای فرمان A، پایه PC7 ابتدا صفر و سپس یک می‌شود. این تغییر یک لبه بالارونده ایجاد می‌کند که به ورودی INT0 میکروکنترلر سنسور اعمال می‌شود. سنسور پس از دریافت این پالس، مقدار دثوکسی هموگلوبین را روی پورت B قرار می‌دهد. برنامه حدود یک میلی ثانیه صبر کرده و سپس داده را در R20 می‌خواند.

A:

```
CBI PORTC, 7
```

```
SBI PORTC, 7
```

```
RCALL Delay
```

```
IN R20, PINB
```

```
CBI PORTC, 7
```

۱۷. رمزگشایی مقدار دثوکسی هموگلوبین

اطلاعاتی که سنسور روی پورت B قرار می‌دهد، با عدد مربوط به شماره دانشجویی رمزگذاری شده است. عدد مورد استفاده در این پروژه 0x34، معادل ۵۲ دهدهی، است. برای بازیابی مقدار اصلی، داده دریافت شده در R20 با 0x34 عملیات XOR می‌شود.

```
LDI R17, 0x34
```

```
EOR R20, R17
```

۱۸. ایجاد پالس برای دریافت اکسی‌هموگلوبین

برای دریافت مقدار اکسی‌هموگلوبین، پایه PD7 ابتدا صفر و سپس یک می‌شود. این لبه بالارونده به ورودی INT1 میکروکنترلر سنسور اعمال می‌شود. پس از تأخیر لازم، مقدار پورت B در رجیستر R21 خوانده شده و پایه PD7 مجدداً صفر می‌شود.

CBI PORTD, 7

SBI PORTD, 7

RCALL Delay

IN R21, PINB

CBI PORTD, 7

۱۹. رمزگشایی مقدار اکسی‌هموگلوبین

مقدار اکسی‌هموگلوبین نیز مانند داده قبلی به صورت رمزگذاری شده دریافت می‌شود. چون مقدار 0x34 از قبل در R17 قرار دارد، با اجرای دستور EOR داده اصلی اکسی‌هموگلوبین بازیابی می‌شود. در پایان این قسمت، مقدار دئوکسی‌هموگلوبین در R20 و مقدار اکسی‌هموگلوبین در R21 قرار دارند.

EOR R21, R17

۲۰. محاسبه، ذخیره و نمایش مقدار SpO₂

پس از دریافت دو داده سنسور، تابع SpO₂ مقدار درصد اشباع اکسیژن را محاسبه می‌کند و نتیجه را در R22 قرار می‌دهد. سپس تابع Save مقدار محاسبه شده را در خانه انتخاب شده EEPROM ذخیره می‌کند. در ادامه تابع Display مقدار را روی دو سون سگمنت نمایش داده و برنامه به حلقه اصلی بازمی‌گردد.

RCALL SpO2

RCALL Save

; spo dar R22

RCALL Display

RJMP Main

۲۱. اجرای فرمان B

فرمان B برای بازیابی یک مقدار ذخیره شده استفاده می شود. تابع Load شماره خانه موجود در R18 را به آدرس EEPROM تبدیل کرده و مقدار آن را در R22 قرار می دهد. سپس تابع Display مقدار بازیابی شده را روی سون سگمنت های دهگان و یکان نمایش می دهد.

B:

RCALL Load

; spo dar R22

RCALL Display

RJMP Main

تابع اسکن صفحه کلید

۲۲. اسکن ردیف اول صفحه کلید

برای بررسی ردیف اول، مقدار 0x7F روی پورت A قرار داده می‌شود. در این حالت پایه PA7 صفر و سایر ردیف‌ها یک هستند. پس از دو دستور NOP برای پایدارشدن وضعیت، مقدار پورت A خوانده می‌شود. چهار بیت پایینی بررسی می‌شوند و در صورت صفرشدن یکی از آن‌ها، فشرده‌شدن یک کلید تشخیص داده می‌شود.

Keypad:

; radif 1

LDI R16, 0x7F

OUT PORTA, R16

NOP

NOP

IN R16, PINA

MOV R17, R16

ANDI R17, 0x0F

CPI R17, 0x0F

BRNE Find

۲۳. اسکن ردیف دوم صفحه کلید

برای اسکن ردیف دوم مقدار 0xBF روی پورت A قرار می‌گیرد. در این حالت پایه PA6 صفر و سایر ردیف‌ها یک هستند. پس از خواندن پورت، فقط چهار بیت پایینی در نظر گرفته می‌شوند. اگر این چهار بیت برابر 0x0F نباشند، یعنی یکی از کلیدهای ردیف دوم فشرده شده است.

; radif 2

LDI R16, 0xBF

OUT PORTA, R16

NOP

NOP

IN R16, PINA

MOV R17, R16

ANDI R17, 0x0F

CPI R17, 0x0F

BRNE Find

۲۴. اسکن ردیف سوم صفحه کلید

در این مرحله مقدار 0xDF روی پورت A قرار داده می شود تا پایه PA5 صفر شود. سپس ستون های صفحه کلید از چهار بیت پایینی پورت خوانده می شوند. اگر یکی از ستون ها صفر شده باشد، برنامه متوجه می شود که کلیدی از ردیف سوم فشرده شده و به بخش Find منتقل می شود.

; radif 3

LDI R16, 0xDF

OUT PORTA, R16

NOP

NOP

IN R16, PINA

MOV R17, R16

ANDI R17, 0x0F

CPI R17, 0x0F

BRNE Find

۲۵. اسکن ردیف چهارم صفحه کلید

برای بررسی ردیف چهارم مقدار 0xEF روی پورت A قرار می‌گیرد. در این وضعیت پایه PA4 صفر می‌شود. مانند ردیف‌های قبلی، چهار بیت پایینی پورت بررسی شده و در صورت تشخیص کلید، برنامه به بخش Find می‌رود. کلیدهای C، D، E و F در این ردیف قرار دارند.

; radif 4

LDI R16, 0xEF

OUT PORTA, R16

NOP

NOP

IN R16, PINA

MOV R17, R16

ANDI R17, 0x0F

CPI R17, 0x0F

BRNE Find

۲۶. حالت عدم فشردن کلید

اگر هیچ کلیدی در چهار ردیف تشخیص داده نشود، مقدار 0xFF در R16 قرار می‌گیرد. قبل از خروج از تابع، تمام پایه‌های ردیف با قراردادن 0xFF روی پورت A به حالت یک بازگردانده می‌شوند. مقدار 0xFF در برنامه اصلی به عنوان نشانه عدم فشردن کلید استفاده می‌شود.

; hichi

LDI R17, 0xFF

OUT PORTA, R17

LDI R16, 0xFF

RET

۲۷. بخش Find

هنگامی که یک کلید تشخیص داده می‌شود، مقدار خام وضعیت ردیف و ستون در R16 باقی می‌ماند. در بخش Find فقط تمام ردیف‌های صفحه کلید به حالت یک بازگردانده می‌شوند. سپس تابع خاتمه می‌یابد و مقدار خام کلید برای تبدیل توسط EEPROM به برنامه اصلی تحویل داده می‌شود.

Find:

LDI R17, 0xFF

OUT PORTA, R17

RET

توابع مربوط به حافظه EEPROM

۲۸. تابع تبدیل مقدار خام صفحه کلید

در این تابع ابتدا بررسی می‌شود که عملیات نوشتن قبلی EEPROM به پایان رسیده باشد. سپس مقدار خام موجود در R16 به عنوان آدرس پایین EEPROM در EEARL قرار می‌گیرد و بخش بالایی آدرس صفر می‌شود. با یک‌کردن بیت EERE عملیات خواندن انجام شده و مقدار واقعی کلید از EEDR وارد R16 می‌شود.

E2prom:

SBIC EECR, EEWE

RJMP E2prom

OUT EEARL, R16

CLR R17

OUT EEARH, R17

SBI EECR, EERE

IN R16, EEDR

RET

۲۹. تابع Readkey و خواندن از بافر SRAM

در این تابع ابتدا بررسی می‌شود که اشاره گر Z در ابتدای بافر قرار دارد یا خیر. اگر بافر خالی باشد، مقدار 0xFF بازگردانده می‌شود. در غیر این صورت اشاره گر Z یک واحد کاهش یافته و آخرین مقدار ذخیره شده از SRAM خوانده می‌شود. بنابراین بافر در این برنامه به صورت پشته یا LIFO عمل می‌کند.

Readkey:

LDI R17, LOW(0x0400)

CP ZL, R17

LDI R17, HIGH(0x0400)

CPC ZH, R17

BREQ Empty

SBIW ZL, 1

LD R16, Z

RET

۳۰. حالت خالی بودن بافر

اگر رجیستر Z دقیقاً برابر آدرس اولیه بافر باشد، یعنی هیچ اطلاعاتی در بافر وجود ندارد. در این حالت برنامه به بخش Empty منتقل می‌شود. مقدار 0xFF در R16 قرار گرفته و تابع خاتمه می‌یابد.

Empty:

LDI R16, 0xFF

RET

تابع تأخیر

۳۱. ایجاد تأخیر حدود یک میلی ثانیه

برای ایجاد فرصت کافی جهت آماده شدن داده سنسور، یک تأخیر نرم افزاری ایجاد شده است. مقدار ۲۵۰ در زوج رجیسترهای R24:R25 قرار می گیرد. در هر بار اجرای حلقه، این مقدار یک واحد کاهش می یابد تا به صفر برسد. با فرکانس یک مگاهرتز، زمان این تابع تقریباً برابر یک میلی ثانیه است.

Delay:

LDI R24, LOW(250)

LDI R25, HIGH(250)

DelayLoop:

SBIW R24, 1

BRNE DelayLoop

RET

تابع محاسبه SpO_2

۳۲. محاسبه مخرج فرمول

فرمول مورد استفاده در برنامه به صورت درصد اکسی هموگلوبین نسبت به مجموع اکسی هموگلوبین و دئوکسی هموگلوبین است. مقدار دئوکسی هموگلوبین از R20 به R23 منتقل شده و مقدار اکسی هموگلوبین R21 به آن اضافه می شود. چون مجموع ممکن است بیشتر از ۲۵۵ شود، بیت نقلی در R26 ذخیره می شود تا مخرج به صورت ۱۶ بیتی تشکیل شود.

SpO2:

; makhraj dar R26:R23

MOV R23, R20

CLR R26

ADD R23, R21

BRCC MakhrajReady

INC R26

۳۳. بررسی صفر نبودن مخرج

پس از تشکیل مخرج ۱۶ بیتی، دو بخش آن با یکدیگر OR می‌شوند. اگر نتیجه صفر باشد، یعنی هر دو مقدار اکسی‌هموگلوبین و دئوکسی‌هموگلوبین صفر بوده‌اند. در این شرایط تقسیم امکان‌پذیر نیست و برنامه به بخش Zero منتقل می‌شود تا نتیجه صفر در نظر گرفته شود.

MakhrajReady:

MOV R16, R23

OR R16, R26

BREQ Zero

۳۴. محاسبه صورت کسر

برای تبدیل نتیجه به درصد، مقدار اکسی‌هموگلوبین در عدد ۱۰۰ ضرب می‌شود. دستور MUL حاصل ضرب ۱۶ بیتی را در زوج رجیسترهای R1:R0 قرار می‌دهد. سپس بخش پایین و بالای حاصل به ترتیب به R24 و R25 منتقل می‌شوند. رجیستر R22 نیز برای نگهداری خارج قسمت صفر می‌شود.

; soorat dar R1:R0

LDI R16, 100

MUL R21, R16

MOV R24, R0

MOV R25, R1

CLR R1

CLR R22

۳۵. انجام تقسیم با روش تفریق متوالی

تقسیم در این برنامه با استفاده از تفریق متوالی انجام می‌شود. صورت کسر در R25:R24 و مخرج در R26:R23 قرار دارند. تا زمانی که صورت از مخرج کوچک‌تر نشده است، مخرج از صورت کم شده و رجیستر R22 یک واحد افزایش پیدا می‌کند. در پایان مقدار R22 برابر قسمت صحیح SpO_2 خواهد بود.

Tghsim:

; moghayese soorat va makhraj

CP R24, R23

CPC R25, R26

; soorat < makhraj

BRLO Done

; soorat >= makhraj

SUB R24, R23

SBC R25, R26

INC R22

RJMP Tghsim

۳۶. مدیریت حالت مخرج صفر و پایان محاسبه

اگر مجموع اکسی هموگلوبین و دئوکسی هموگلوبین صفر باشد، برنامه مقدار نتیجه را نیز صفر قرار می‌دهد. در حالت عادی، پس از پایان تقسیم برنامه مستقیماً به بخش Done می‌رسد. سپس مقدار نهایی SpO_2 در R22 باقی مانده و تابع خاتمه می‌یابد.

Zero:

CLR R22

Done:

RET

تابع ذخیره‌سازی SpO_2

۳۷. تبدیل شماره خانه به آدرس EEPROM

شماره خانه انتخاب‌شده در رجیستر R18 قرار دارد و بین ۱ تا ۸ است. برای تبدیل آن به آدرس‌های 0x10 تا 0x17، مقدار 0x0F به شماره خانه اضافه می‌شود. بنابراین خانه اول به آدرس 0x10 و خانه هشتم به آدرس 0x17 تبدیل می‌شود.

Save:

```
MOV R16, R18
```

```
LDI R17, 0x0F
```

```
ADD R16, R17
```

۳۸. نوشتن مقدار SpO_2 در EEPROM

قبل از نوشتن، برنامه منتظر می‌ماند تا عملیات قبلی EEPROM پایان یابد. سپس آدرس خانه در رجیسترهای آدرس EEPROM قرار گرفته و مقدار SpO_2 موجود در R22 به EEDR منتقل می‌شود. برای شروع نوشتن، ابتدا بیت EEMWE و سپس بیت EEWE یک می‌شوند.

E2promWrite:

```
SBIC EECR, EEWE
```

```
RJMP E2promWrite
```

```
OUT EEARL, R16
```

```
CLR R17
```

```
OUT EEARH, R17
```

OUT EEDR, R22

SBI EECR, EEMWE

SBI EECR, EEWE

RET

تابع بازیابی SpO_2

۳۹. تبدیل شماره خانه برای خواندن

در تابع Load نیز شماره خانه موجود در R18 با مقدار 0x0F جمع می‌شود. به این ترتیب شماره‌های ۱ تا ۸ به آدرس‌های 0x10 تا 0x17 EEPROM تبدیل می‌شوند. این آدرس همان محلی است که قبلاً تابع Save مقدار SpO_2 را در آن ذخیره کرده است.

Load:

MOV R16, R18

LDI R17, 0x0F

ADD R16, R17

۴۰. خواندن مقدار ذخیره شده از EEPROM

ابتدا برنامه منتظر پایان عملیات نوشتن احتمالی EEPROM می‌ماند. سپس آدرس موردنظر در رجیسترهای آدرس EEPROM قرار می‌گیرد. با یک‌کردن بیت EERE عملیات خواندن انجام می‌شود. مقدار خوانده شده از EEDR در رجیستر R22 قرار می‌گیرد تا برای نمایش استفاده شود.

eeWait:

SBIC EECR, EEWE

RJMP eeWait

OUT EEARL, R16

CLR R17

OUT EEARH, R17

SBI EECR, EERE

IN R22, EEDR

RET

تابع نمایش روی سون سگمنت

۴۱. جداسازی دهگان و یکان

مقدار SpO_2 از R22 به R24 منتقل می شود و رجیستر R23 برای شمارش دهگان صفر می شود. در حلقه Dahgan تا زمانی که مقدار R24 حداقل ۱۰ باشد، عدد ۱۰ از آن کم شده و R23 یک واحد افزایش می یابد. در پایان، R23 شامل رقم دهگان و R24 شامل رقم یکان است.

Display:

MOV R24, R22

CLR R23

Dahgan:

CPI R24, 10

BRLO Show7Seg

SUBI R24, 10

INC R23

RJMP Dahgan

۴۲. نمایش رقم یکان و دهگان

ابتدا رقم یکان از R24 به R16 منتقل شده و تابع SegCode الگوی مناسب سون سگمنت را در R17 قرار می دهد. این الگو روی پورت C نوشته می شود. سپس همین عملیات برای رقم دهگان موجود در R23 انجام شده و نتیجه روی پورت D قرار می گیرد.

Show7Seg:

; R23 dahgan va R24 yekan

MOV R16, R24

RCALL SegCode

OUT PORTC, R17

MOV R16, R23

RCALL SegCode

OUT PORTD, R17

RET

تابع تبدیل رقم به کد سون سگمنت

۴۳. تشخیص رقم مورد نظر

در تابع SegCode مقدار رقم موجود در R16 به ترتیب با اعداد صفر تا ۹ مقایسه می شود. در صورت برابر بودن، برنامه به برچسب مربوط به همان رقم منتقل می شود. اگر مقدار خارج از محدوده صفر تا ۹ باشد، R17 صفر می شود و هیچ سگمنتی روشن نخواهد شد.

SegCode:

CPI R16, 0

BREQ Seg0

CPI R16, 1

BREQ Seg1

CPI R16, 2

BREQ Seg2

CPI R16, 3

BREQ Seg3

CPI R16, 4

BREQ Seg4

CPI R16, 5

BREQ Seg5

CPI R16, 6
BREQ Seg6

CPI R16, 7
BREQ Seg7

CPI R16, 8
BREQ Seg8

CPI R16, 9
BREQ Seg9

CLR R17
RET

۴۴. کد نمایش ارقام صفر تا چهار

سون سگمنت‌های استفاده شده از نوع کاتد مشترک هستند؛ بنابراین یک منطقی باعث روشن شدن هر سگمنت می‌شود. بیت‌های صفر تا شش به ترتیب مربوط به سگمنت‌های a تا g هستند. مقادیر زیر الگوی لازم برای نمایش ارقام صفر تا چهار را تولید می‌کنند.

Seg0:

```
LDI R17, 0x3F
RET
```

Seg1:

```
LDI R17, 0x06
RET
```

Seg2:

```
LDI R17, 0x5B
RET
```

Seg3:

```
LDI R17, 0x4F
RET
```

Seg4:

```
LDI R17, 0x66
RET
```

۴۵. کد نمایش ارقام پنج تا نه

در ادامه الگوهای مربوط به ارقام پنج تا نه در رجیستر R17 قرار می‌گیرند. این مقادیر با فرض اتصال بیت صفر پورت به سگمنت a و بیت شش به سگمنت g نوشته شده‌اند. بیت هفتم در تمام این کدها صفر است تا پایه‌های PC7 و PD7 پس از نمایش در سطح صفر قرار گیرند.

Seg5:

```
LDI R17, 0x6D
RET
```

Seg6:

```
LDI R17, 0x7D
RET
```

Seg7:

```
LDI R17, 0x07
RET
```

Seg8:

```
LDI R17, 0x7F
RET
```

Seg9:

```
LDI R17, 0x6F
RET
```

بخش اولیه حافظه EEPROM

۴۶. آغاز بخش EEPROM

با دستور ESEG بخش مربوط به حافظه EEPROM آغاز می‌شود. در این قسمت جدول تبدیل مقادیر خام صفحه کلید به کد واقعی کلیدها ذخیره شده است. هر دستور .ORG آدرس EEPROM را مشخص کرده و دستور DB مقدار موردنظر را در آن آدرس قرار می‌دهد.

.ESEG

۴۷. جدول تبدیل کلیدهای صفر تا سه

مقادیر خام مربوط به چهار کلید ردیف اول صفحه کلید در آدرس‌های مشخص EEPROM ذخیره شده‌اند. هنگامی که یکی از این کلیدها فشرده شود، مقدار خام آن به عنوان آدرس EEPROM استفاده شده و کد واقعی عدد از حافظه خوانده می‌شود.

.ORG 0b01110111

.DB 0x03

.ORG 0b01111011

.DB 0x02

.ORG 0b01111101

.DB 0x01

.ORG 0b01111110

.DB 0x00

۴۸. جدول تبدیل کلیدهای چهار تا هفت

این بخش مربوط به چهار کلید ردیف دوم صفحه کلید است. هر مقدار خام به کد عددی مناسب تبدیل می‌شود. برای مثال مقدار خام 0xBE به عدد چهار و مقدار خام 0xB7 به عدد هفت تبدیل می‌شود.

.ORG 0b10110111

.DB 0x07

.ORG 0b10111011

.DB 0x06

.ORG 0b10111101

.DB 0x05

.ORG 0b10111110

.DB 0x04

۴۹. جدول تبدیل کلیدهای هشت، نه، A و B

مقادیر مربوط به ردیف سوم صفحه کلید در این بخش قرار دارند. کلیدهای A و B به ترتیب با کدهای 0x0A و 0x0B ذخیره شده‌اند. برنامه با استفاده از این کدها تشخیص می‌دهد که کاربر فرمان ثبت نمونه جدید یا بازیابی داده قبلی را وارد کرده است.

.ORG 0b11010111

.DB 0x0B

.ORG 0b11011011

.DB 0x0A

.ORG 0b11011101

.DB 0x09

.ORG 0b11011110

.DB 0x08

۵۰. جدول تبدیل کلیدهای C ، D ، E و F

در این قسمت مقادیر مربوط به ردیف چهارم صفحه کلید تعریف شده‌اند. کد کلید F برابر 0x0F است و برای اعلام پایان فرمان استفاده می‌شود. کلیدهای C ، D و E نیز در جدول وجود دارند، هرچند در عملکرد اصلی این پروژه استفاده نمی‌شوند.

.ORG 0b11100111

.DB 0x0F

.ORG 0b11101011

.DB 0x0E

.ORG 0b11101101

.DB 0x0D

.ORG 0b11101110

.DB 0x0C

خلاصه:

این برنامه ابتدا فرمان کاربر را از صفحه کلید ماتریسی دریافت می‌کند. در فرمان A ، میکروکنترلر با ایجاد دو پالس، مقادیر اکسی‌هموگلوبین و دئوکسی‌هموگلوبین را از سنسور دریافت و با عدد 0x34 رمزگشایی می‌کند. سپس مقدار SpO_2 محاسبه شده، در یکی از هشت خانه EEPROM ذخیره و روی دو سون‌سگمنت نمایش داده می‌شود. در فرمان B ، مقدار ذخیره‌شده از خانه انتخابی EEPROM خوانده شده و بدون دریافت مجدد اطلاعات سنسور روی نمایشگر نشان داده می‌شود

شماتیک مدار در شبیه سازی:

